Financial Assistant Agent

Full Technical Documentation

How the system works, how files connect, and why decisions were made

FastAPI	Gradio 6	Llama Q2_K	FAISS + RAG
Backend	Frontend	Local LLM	Knowledge

2026 — Python • spaCy • llama-cpp-python • SentenceTransformers

Created by Ndumiso Khwela with assistance from Thando Ngcobo

1. Project Overview

We built FinancialAssistantAI as a locally-running personal finance assistant. The idea was simple: We wanted a conversational AI that could give budgeting, debt, and investment advice without sending my financial questions to a cloud service. Everything runs on my own machine — no API keys, no subscription fees, no data leaving the device.

The system uses a quantised Llama language model as its brain, a FastAPI server as its backbone, a Gradio interface as its face, and a RAG (Retrieval-Augmented Generation) pipeline to keep answers grounded in real financial knowledge rather than just the model's training data.

1.1 What It Does End-to-End

Here is the complete journey of a single user message through the system:

1. User types a question into the Gradio chat interface (port 7860)
2. Gradio sends an HTTP POST request to the FastAPI backend (port 8000)
3. FastAPI receives the request and passes it to the Orchestrator
4. The Orchestrator reads the message, detects keywords, and routes to the right agent
5. The selected agent builds a structured prompt combining: the user's message + relevant context from the RAG knowledge base + conversation memory from previous turns
6. The completed prompt is sent to the LLM Engine, which runs the local Llama model
7. The LLM generates a response (30-90 seconds on CPU)
8. The response travels back through the agent → orchestrator → FastAPI → HTTP → Gradio
9. Gradio displays the response in the chat window
10. The full exchange is saved to SQLite for memory across sessions

1.2 Technology Choices and Why

Technology	Role	Why I chose it
FastAPI	Backend API server	Async-native, automatic validation via Pydantic, clean REST endpoint definitions
Gradio 6.x	Chat UI	Minimal setup for a chat interface; built-in support for multi-tab layouts
llama-cpp-python	LLM inference	Runs GGUF quantised models on CPU — no GPU required
finance-Llama3-8B Q2_K	Language model	Finance-domain fine-tuned Llama; Q2_K quantisation fits in ~2.7 GB RAM
Technology	**Role**	**Why I chose it**
SentenceTransformers	RAG embedding	all-MiniLM-L6-v2 is fast (~90 MB) and produces good semantic embeddings
FAISS	Vector search	CPU-optimised nearest-neighbour search for retrieving relevant document chunks
SQLite + aiosqlite	Persistence	Zero-config local database; aiosqlite makes it asyncsafe with FastAPI
Pydantic	Data validation	Type-safe request/response models with automatic error messages

2. Complete File Structure

Every file in the project has a specific job. Nothing is redundant — even the utility scripts exist for a reason.

```
FinancialAssistantAI/
│
├── agents/                     ← The four specialist agents
│   ├── __init__.py             ← Exports all agents for easy import
│   ├── budget_advisor.py       ← Budgeting and expense analysis
│   ├── debt_coach.py           ← Debt repayment strategies
│   ├── portfolio_agent.py      ← Investment and asset allocation
│   └── sandbox_executor.py     ← Safe financial calculations
│
├── app/                        ← Request/response infrastructure
│   ├── logger.py               ← Logs every interaction to logs/chat.log
│   ├── models.py               ← Pydantic data models (ChatRequest, ChatResponse)
│   ├── prompt_service.py       ← Loads prompt templates from prompts/
│   └── retrieval_service.py    ← Early FAISS RAG implementation (legacy)
│
├── core/                       ← The engine room
│   ├── orchestrator.py         ← Routes requests to the right agent
│   ├── llm_engine.py           ← Loads and runs the local Llama model
│   ├── rag_service.py          ← Production RAG with timeout + cache
│   ├── memory_service.py       ← Per-session conversation history
│   └── database_service.py     ← SQLite async persistence layer
│
├── ui/
│   └── app.py                  ← Gradio 6.x chat interface (5 tabs)
│
├── utils/
│   ├── __init__.py
│   ├── config.py               ← Utils-level config helpers (validators)
│   └── data_loader.py          ← Loads docs/ knowledge base for RAG indexing
│
├── docs/                       ← RAG knowledge base (plain text documents)
├── prompts/                    ← System prompt templates (one .txt per agent)
├── tests/                      ← Unit tests for agents and core services
├── logs/                       ← Auto-created; stores chat.log
├── models/                     ← Auto-created by downloadModel.py; stores .gguf
```

```
│
├── config.py                  ← Single source of truth for all settings
├── main.py                    ← FastAPI app — defines all API endpoints
├── run_backend.py             ← Starts the FastAPI server with uvicorn
├── run_frontend.py            ← Starts the Gradio UI
├── finance_chat_Term.py       ← Terminal-only chat (no UI, for quick testing)
├── finance_chat.py            ← Original prototype (single-file Gradio + LLM)
├── downloadModel.py           ← Downloads quantised Llama from Hugging Face
├── kill_ports.py              ← Clears ports 7860/8000 if stuck
├── install.py                 ← Full installation script
├── setup.py                   ← Package setup
└── requirements.txt           ← All Python dependencies
```

3. config.py — The Single Source of Truth

config.py is where everything is controlled from. Every other file imports settings from here — nothing is hardcoded anywhere else. This means to change any behaviour (timeout, model path, port, token limit), I only ever edit one file.

3.1 What It Contains

Setting	Value	What it controls
MODEL_PATH	models/finance-Llama3-8B.Q2_K.gguf	Path to the quantised model file on disk
MODEL_N_CTX	512	Context window size — lower = faster token processing on CPU
MODEL_MAX_TOKENS	150	Maximum tokens the LLM generates per response
MODEL_N_THREADS	8	CPU threads for inference — set to your core count
MODEL_TEMPERATURE	0.7	Creativity of responses (0=deterministic, 1=creative)
REQUEST_TIMEOUT	300	Seconds before a request times out (5 min for slow CPU)
BACKEND_HOST	127.0.0.1	FastAPI host — localhost only, not exposed to network

Setting	Value	What it controls
BACKEND_PORT	8000	FastAPI port
BACKEND_URL	http://127.0.0.1:8000	Full URL the frontend uses to reach the backend
FRONTEND_PORT	7860	Gradio port
DB_PATH	financial_assistant.db	SQLite database file location
RAG_TOP_K	3	Number of document chunks retrieved per query
RAG_TIMEOUT	8	Seconds before RAG encode is skipped to prevent hanging

3.2 Why These Values Matter

MODEL_MAX_TOKENS = 150: The most important CPU performance setting. The financeLlama3-8B model generates 2–5 tokens per second on CPU. At 150 tokens max, the worstcase response time is 75 seconds — safely within the 300s timeout. At the original 512 tokens, responses could take over 4 minutes, causing the request to time out before finishing.

MODEL_N_CTX = 512: The context window determines how much text the model processes at once. Smaller windows process faster. 512 is enough for a question + RAG context + a few conversation turns.

RAG_TIMEOUT = 8: Embedding a query with SentenceTransformers takes 3–8 seconds on CPU. If encoding takes longer than 8 seconds, RAG is skipped and the LLM answers from training knowledge alone. This prevents the entire request from hanging just because the RAG encode was slow.

4. Startup — How the System Comes Alive

There are two processes that need to run simultaneously: the backend and the frontend. They are started in separate terminal windows.

4.1 run_backend.py

Purpose: Starts the FastAPI server using uvicorn.

```
# run_backend.py
import uvicorn
import config

if __name__ == '__main__':
    uvicorn.run('main:app', host=config.BACKEND_HOST,
                port=config.BACKEND_PORT, reload=False)
```

When this runs, three things happen in sequence:

1. uvicorn imports main.py and creates the FastAPI app instance
2. The LLM Engine initialises — it loads the Llama model from disk into RAM (~2.7 GB). This takes 15–30 seconds on first start.
3. The RAG Service initialises — it loads the docs/ knowledge base, chunks the documents, and builds the FAISS vector index. This takes 5–15 seconds.

NOTE The model loading happens once at startup and stays in memory. Subsequent requests do not reload the model — they reuse the already-loaded instance. This is why the first request is slow but later ones are faster.

4.2 run_frontend.py

Purpose: Starts the Gradio UI in a separate process.

```
# run_frontend.py
import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ui.app import demo

if __name__ == '__main__':
    demo.launch(server_port=config.FRONTEND_PORT,
                server_name='127.0.0.1')
```

When Gradio starts, it immediately calls check_backend() to verify the FastAPI server is reachable. The status indicator in the UI ("OK" or "DEGRADED") reflects this check. If the backend is not running, the UI still loads but all requests will fail with "Cannot connect to backend."

4.3 Why Two Separate Processes?

The backend and frontend run separately for two reasons. First, the LLM is a blocking operation — it takes 30–90 seconds to generate a response. If the LLM ran inside the same process as

the Gradio UI, the entire UI would freeze during generation. Running them as separate processes with HTTP communication means Gradio stays responsive while the backend is busy. Second, this separation means the backend could be replaced with a different frontend (terminal, web app, API client) without changing any backend code.

5. main.py — The API Layer

main.py defines the FastAPI application and every HTTP endpoint the frontend can call. It is the single entry point for all requests from the UI.

5.1 API Endpoints

Method	Endpoint	What it does	Called by
GET	/health	Returns server status and model load state	UI status check on startup
POST	/query	General financial question — routed by orchestrator	Main chat tab
POST	/agent/budget	Direct route to Budget Advisor agent	Budget tab
POST	/agent/debt	Direct route to Debt Coach agent	Debt tab
POST	/agent/portfolio	Direct route to Portfolio Manager agent	Portfolio tab
POST	/agent/sandbox	Direct route to Sandbox Executor	Calculator tab
GET	/memory/{session_id}	Returns conversation history for a session	UI on load
DELETE	/memory/{session_id}	Clears conversation history for a session	Clear chat button

5.2 Request/Response Flow

Every POST request to /query or the agent endpoints follows the same pattern:

```
# Incoming request shape (models.py — ChatRequest)
{
    'user_id':    'user_abc123',  # session identifier
    'message':    'How do I pay off my credit card faster?',
```

```
    'session_id': 'session_xyz'   # for memory lookup
}

# Outgoing response shape (models.py — ChatResponse)
{
    'response': 'The two most effective strategies are...',
    'agent':    'debt_coach',  # which agent handled it
    'sources':  ['debt_strategies.txt']  # RAG sources used
}
```

5.3 Pydantic Models (app/models.py)

app/models.py defines the data contracts for every request and response. FastAPI uses these to automatically validate incoming data — if a request is missing a required field or has the wrong type, FastAPI rejects it with a clear error before any business logic runs. This means the agents never receive malformed data.

6. core/orchestrator.py — The Brain of Routing

orchestrator.py is the central coordinator. When a request arrives at /query, the orchestrator decides which agent handles it, fetches relevant context from RAG, retrieves conversation memory, assembles the final prompt, and coordinates the LLM call.

6.1 Agent Auto-Detection

The orchestrator reads the user's message and matches it against keyword lists for each agent:

```
AGENT_KEYWORDS = {
    'budget':
['budget','income','expenses','spending','saving','salary','monthly'],
    'debt':      ['debt','loan','credit card','interest
rate','repay','owe','balance'],
    'portfolio':
['invest','stocks','portfolio','asset','diversif','risk','return'],
    'sandbox':   ['calculate','compute','formula','how much','what
if','compound'],
}

def detect_agent(message: str) -> str:
    msg = message.lower()
    for agent, keywords in AGENT_KEYWORDS.items():
```

```
        if any(kw in msg for kw in keywords):
            return agent
    return 'general'  # fallback to direct LLM answer
```

The first keyword match wins. If no keywords match, the question goes to the general handler — the LLM answers directly with RAG context but without a specialised agent system prompt.

6.2 Request Processing Pipeline

Once the agent is detected, the orchestrator runs these steps in order:

1. Load conversation memory for the session_id from memory_service
2. Call rag_service.retrieve_docs(message) to get the top-3 relevant document chunks
3. Call the detected agent's build_prompt() method to assemble the final prompt
4. Pass the prompt to llm_engine.generate() and await the response
5. Save the new exchange (user message + AI response) to memory_service
6. Log the interaction via app/logger.py to logs/chat.log
7. Persist the exchange to the SQLite database via database_service
8. Return the response to main.py which sends it back as HTTP JSON

6.3 Why the Orchestrator Exists Separately

Originally the project had two separate FastAPI apps — one in main.py and one that grew inside orchestrator.py. The orchestrator now has no awareness of HTTP at all — it only knows about agents, RAG, memory, and the LLM. main.py handles all HTTP concerns. This separation means I could swap Gradio for a different frontend, or swap FastAPI for a different framework, without touching any of the core logic.

7. agents/ — The Four Specialist Agents

Each agent is a Python class with a consistent interface. They all have a build_prompt() method that receives the user message, conversation history, and RAG context — and returns a fully assembled prompt string ready for the LLM. The agents never call the LLM directly; they only build prompts.

7.1 Budget Advisor (agents/budget_advisor.py)

The Budget Advisor analyses income versus expenses and gives actionable budgeting advice. It implements the 50/30/20 rule as its analytical framework.

Component	Detail
Class	BudgetAdvisorAgent
Data model	BudgetProfile dataclass: monthly_income, expenses dict, savings_goal_pct, currency
Framework	50/30/20 rule — 50% needs, 30% wants, 20% savings
Benchmarks	BUDGET_BENCHMARKS dict: housing 30%, food 10%, transport 15%, utilities 5%, etc.
Key methods	analyse_budget() — computes variance from benchmarks; build_prompt() — formats for LLM
Triggered by	Keywords: budget, income, expenses, spending, saving, salary, monthly

When a user mentions their income and expenses, the agent calculates how each spending category compares to the benchmark. If housing takes 40% of income instead of the recommended 30%, the agent surfaces this and explains why it matters. The LLM then turns this structured analysis into a conversational response.

7.2 Debt Coach (agents/debt_coach.py)

The Debt Coach helps users understand, prioritise, and eliminate their debts. It implements two widely-used strategies: Avalanche (highest interest rate first) and Snowball (smallest balance first).

Component	Detail
Class	DebtCoachAgent
Data models	Debt dataclass (name, balance, interest_rate, minimum_payment, currency); RepaymentPlan dataclass
Avalanche strategy	Order debts by interest rate descending — mathematically optimal, saves the most money
Snowball strategy	Order debts by balance ascending — psychologically motivating, faster early wins
Calculated outputs	Monthly interest per debt, total interest paid, months to payoff, month-bymonth schedule

Triggered by	Keywords: debt, loan, credit card, interest rate, repay, owe, balance

```
# Debt dataclass — represents a single debt account
@dataclass
class Debt:
    name: str
    balance: float           # current outstanding balance
    interest_rate: float     # annual rate as percentage e.g. 18.5
    minimum_payment: float   # required minimum monthly payment
    currency: str = 'USD'

    @property
    def monthly_interest(self) -> float:
        return self.balance * (self.interest_rate / 100 / 12)
```

7.3 Portfolio Manager (agents/portfolio_agent.py)

The Portfolio Manager covers investment advice, asset allocation, diversification, and risk assessment. It uses predefined risk profiles to match users to appropriate investment approaches.

Component	Detail
Class	PortfolioAgent
Data models	Holding dataclass (ticker, name, value, asset_class); PortfolioAnalysis dataclass
Risk profiles	RISK_PROFILES dict: conservative, moderate, aggressive — each with target allocations
Asset classes	Equities, bonds, cash, real estate, commodities — tracked per holding
Analysis	Computes current allocation percentages, compares to target profile, identifies drift
Triggered by	Keywords: invest, stocks, portfolio, asset, diversif, risk, return

7.4 Sandbox Executor (agents/sandbox_executor.py)

The Sandbox Executor handles calculation requests safely. Financial formulas like compound interest, loan amortisation, and future value calculations can be executed programmatically rather than relying on the LLM to do arithmetic (which LLMs do unreliably).

Component	Detail
Class	SandboxExecutorAgent
Safety model	Uses Python ast.literal_eval and a whitelist of allowed operations — no eval() of arbitrary code
Built-in formulas	compound_interest(), loan_payment(), future_value(), present_value(), roi()
Error handling	Invalid expressions return a clear error message rather than crashing
Triggered by	Keywords: calculate, compute, formula, how much, what if, compound

WHY SANDBOX? LLMs are notoriously bad at arithmetic. Asking 'what is the compound interest on R5,000 at 8% for 10 years?' will give different (sometimes wrong) answers each time. The Sandbox Executor computes the answer programmatically and passes the correct result to the LLM to explain — guaranteeing numerical accuracy.

7.5 agents/__init__.py — The Export Hub

agents/__init__.py imports all four agents and re-exports them under clean names. This means any other file in the project can write:

```
from agents import BudgetAdvisorAgent, DebtCoachAgent, PortfolioAgent,
SandboxExecutorAgent
```

instead of needing to know the specific file each class lives in. This is a small but important design detail — if an agent is ever moved to a different file, only __init__.py needs updating, not every file that imports it.

8. core/ — The Engine Room

The core/ folder contains the services that every agent depends on. These are not agents themselves — they are infrastructure that agents use.

8.1 core/llm_engine.py — Local LLM Inference

llm_engine.py is responsible for loading the Llama model and running inference. It is one of the most performance-critical files in the project because it handles the slowest operation.

Key design decisions:

- Model loaded once at startup — the Llama class is instantiated when llm_engine.py is first imported. Subsequent calls reuse the same instance. Loading takes 15–30 seconds so it must happen once, not per-request.
- [object Object]the LLM's generate() call is synchronous and blocking. Running it directly inside an async FastAPI endpoint would freeze the entire event loop (blocking all other requests). asyncio.to_thread() offloads the blocking call to a thread pool, keeping the event loop free.
- max_tokens = 150 — the critical CPU performance setting. Lower token limits mean faster responses. 150 tokens is enough for a clear, useful financial answer.
- n_ctx = 512 — smaller context window processes faster on CPU. 512 is enough for prompt + RAG context + recent conversation turns.

```
# The critical async wrapper — prevents the LLM from blocking the event loop
async def generate(self, prompt: str) -> str:
    loop = asyncio.get_event_loop()
    response = await loop.run_in_executor(
        None,  # uses default thread pool executor
        self._generate_sync,  # the blocking function
        prompt
    )
    return response
```

WHY THIS MATTERS Without asyncio.to_thread(), every LLM request would freeze all of FastAPI for 30-90 seconds. No health checks, no other requests, nothing. The frontend would show 'Cannot connect to backend' because even the /health endpoint couldn't respond. This was the root cause of many early connection errors.

8.2 core/rag_service.py — Retrieval-Augmented Generation

rag_service.py implements the RAG pipeline. Instead of relying solely on the LLM's training data, RAG retrieves relevant passages from a curated knowledge base and includes them in the prompt. This grounds the model's answers in factual, up-to-date financial information.

How RAG works step by step:

1. At startup, data_loader.py reads all .txt files from the docs/ folder
2. Each document is split into overlapping chunks (e.g. 200-word chunks with 50-word overlap)
3. Each chunk is embedded using SentenceTransformers (all-MiniLM-L6-v2) — converted to a 384-dimension vector
4. All vectors are stored in a FAISS index in memory

5. When a user asks a question, the question is also embedded into a 384-dimension vector
6. FAISS performs nearest-neighbour search to find the top-3 most similar document chunks
7. Those chunks are prepended to the LLM prompt as context

Two performance optimisations I added to handle my 4GB CPU constraint:

- [object Object]the embedding step runs in a background thread. If it takes longer than 8 seconds (which it can on a busy CPU), RAG is skipped entirely and the LLM answers without context. This prevents requests from hanging on a slow encode step.
- [object Object]the first time a query is embedded, the result is cached. Repeated or similar questions return instantly from cache without re-running the encoder.

```
async def retrieve_docs(self, query: str) -> list[str]:
    # Check cache first
    if query in self._cache:
        return self._cache[query]

    try:
        # Run embedding in thread with timeout
        query_vec = await asyncio.wait_for(
            asyncio.to_thread(self.model.encode, [query]),
            timeout=self.rag_timeout  # 8 seconds
        )
        distances, indices = self.index.search(query_vec, self.top_k)
        results = [self.chunks[i] for i in indices[0] if i >= 0]
        self._cache[query] = results
        return results
    except asyncio.TimeoutError:
        logger.warning('RAG encode timed out — skipping context')
        return []
```

8.3 core/memory_service.py — Conversation Memory

memory_service.py manages per-session conversation history. Without memory, every message would be treated as a standalone question — the assistant would have no idea what was discussed earlier in the conversation.

Memory is stored in two places:

- In-memory (Python dict) — fast access for the current session. The last N turns (configurable) are kept in RAM and included in every prompt.
- SQLite (via database_service) — persistent storage. If the server restarts, memory can be reloaded from the database so conversations are not lost.

The memory is formatted into the prompt as a conversation transcript, prepended before the user's current message so the LLM understands the context of what was said before.

8.4 core/database_service.py — SQLite Persistence

database_service.py handles all database operations using aiosqlite — an async wrapper around Python's built-in sqlite3. Like the LLM engine, all database operations are awaited so they never block the FastAPI event loop.

The database stores:

- Every conversation turn (user message + AI response + timestamp + session_id)
- Which agent handled each message
- Which RAG sources were used

This means I can later query the database to see patterns in what questions are being asked most, which agents are used most, and how response quality changes over time.

8.5 core/orchestrator.py — Coordination

Covered in detail in Section 6. The orchestrator imports and uses all four of the above services: llm_engine, rag_service, memory_service, and database_service. It is the only file in core/ that imports from agents/.

[1]. docs/ and prompts/ — Knowledge and Personality

The docs/ folder contains the plain text documents that the RAG system uses as its knowledge base. These are the documents that get chunked, embedded, and searched when a user asks a question. The quality of these documents directly affects the quality of the assistant's answers.

The knowledge base covers:

- Personal budgeting principles and the 50/30/20 framework

---

[1] .1 docs/ — The RAG Knowledge Base

- Debt repayment strategies — Avalanche method, Snowball method, consolidation options
- Emergency fund guidelines — how much to save, where to keep it
- Investment basics — asset classes, diversification, risk vs return
- Compound interest explanations and examples
- South African financial context — tax brackets, retirement annuities (RA), TFSA rules

Each document is a plain .txt file. The data_loader.py utility reads all files in docs/, strips whitespace, and feeds the text to the RAG service for indexing at startup.

9.2 prompts/ — System Prompt Templates

Each agent has a corresponding system prompt template stored as a .txt file in prompts/. The system prompt defines the agent's personality, constraints, and response format.

Example of what a system prompt template looks like:

```
# prompts/debt_coach.txt
You are a Debt Coach, a specialist in helping people understand and eliminate
their debts. You are empathetic, practical, and non-judgmental.

When analysing debts, always:
- Present both the Avalanche (highest interest first) and Snowball (smallest
  balance first) strategies
- Calculate the total interest paid under each strategy
- Recommend the approach that best suits the user's situation
- Never suggest predatory lending or schemes

Context from knowledge base:
{rag_context}

Conversation so far:
{memory}

User: {message}
Debt Coach:
```

prompt_service.py in the app/ folder loads these templates and fills in the {rag_context}, {memory}, and {message} placeholders. This keeps the prompt text out of the Python code and makes it easy to tune the assistant's behaviour without touching any code.

10. ui/app.py — The Gradio Interface

ui/app.py is the entire frontend of the application. It is a single Python file that defines a Gradio 6.x interface with five tabs, connecting to the FastAPI backend via HTTP.

10.1 The Five Tabs

Tab	Purpose	Backend endpoint
Chat	General financial questions — routed automatically to the right agent	/query
Budget	Structured budget analysis with income/expense inputs	/agent/budget
Debt	Debt repayment calculator with JSON debt input	/agent/debt
Portfolio	Investment portfolio analysis	/agent/portfolio
Calculator	Direct financial calculations and formula execution	/agent/sandbox

10.2 How the Frontend Talks to the Backend

The frontend communicates with the backend via two private HTTP helper functions:

```
BACKEND = config.BACKEND_URL   # 'http://127.0.0.1:8000'
TIMEOUT = config.REQUEST_TIMEOUT  # 300 seconds

def _get(path: str) -> Optional[Dict]:
    r = requests.get(f'{BACKEND}{path}', timeout=10)
    return r.json()


def _post(path: str, payload: Dict) -> Dict:
    try:
        r = requests.post(f'{BACKEND}{path}', json=payload, timeout=TIMEOUT)
        return r.json()
    except requests.exceptions.ConnectionError:
        return {'error': 'Cannot connect to backend. Is the server running?'}
```

Every button click in the UI calls _post() with the appropriate endpoint and payload. The response is then unpacked and displayed in the chat window.

10.3 Backend Health Check

```
def check_backend() -> Tuple[bool, str]:
    data = _get('/health')
    if not data:
        return False, 'Backend offline'
    status = data.get('status', 'unknown')
    icon = 'OK' if status == 'healthy' else 'DEGRADED'
    return status == 'healthy', f'[{icon}] {data}'
```

This runs when Gradio loads and again on each tab switch. The status badge in the UI reflects the result. If the backend is offline, all request buttons show a clear error message instead of hanging indefinitely.

10.4 Gradio Queue Fix (Critical)

Gradio 6.x has an internal default timeout of approximately 60 seconds. Since the LLM takes 30–90 seconds to respond, requests were being silently killed before the response arrived — the UI showed an error even though the backend was working correctly.

The fix was adding demo.queue(max_size=4) before demo.launch():

```
# Without this, Gradio kills requests after ~60 seconds
demo.queue(max_size=4)  # enables proper async queue, removes 60s kill switch
demo.launch(server_port=config.FRONTEND_PORT)
```

With the queue enabled, Gradio uses a proper async queue instead of the default timeout mechanism, and requests stay alive for the full REQUEST_TIMEOUT duration.

11. app/ — Request Infrastructure

The app/ folder contains the supporting infrastructure for the API layer — logging, data models, prompt loading, and the early RAG implementation.

11.1 app/logger.py — Interaction Logging

Every request and response is logged to logs/chat.log with a timestamp, user ID, and both the message and AI response. This log is append-only and human-readable.

```
# Example log entry
2025-03-06T14:32:11 | USER=user_abc | MSG=How do I pay off my credit card? | AI=The
two most effective strategies...
```

This logging serves two purposes: debugging (I can see exactly what happened during a request) and analysis (I can review what kinds of questions users ask most frequently).

11.2 app/models.py — Data Contracts

Pydantic models define exactly what data the API accepts and returns. FastAPI reads these models to generate automatic validation — requests with missing or wrongly-typed fields are rejected before they reach any business logic.

11.3 app/prompt_service.py — Template Loading

prompt_service.py reads .txt files from prompts/ and provides a load_prompt(agent_name) function. Agents call this to get their system prompt template, then fill in the {rag_context}, {memory}, and {message} placeholders before passing the completed prompt to the LLM engine.

11.4 app/retrieval_service.py — Legacy RAG

This was the original FAISS RAG implementation before it was moved and improved in core/rag_service.py. It remains in app/ for reference but core/rag_service.py is the version used in production. The key difference is that retrieval_service.py did not have the async timeout or caching that was added when performance issues emerged on my 4GB CPU machine.

12. Utility Files

12.1 downloadModel.py

Downloads the quantised Llama model from Hugging Face Hub and saves it to the models/ directory. This only needs to run once.

```
from huggingface_hub import hf_hub_download
```

```
file_path = hf_hub_download(
    repo_id='QuantFactory/finance-Llama3-8B-GGUF',
    filename='finance-Llama3-8B.Q2_K.gguf',
    cache_dir='models/'
)
```

Why Q2_K quantisation? The full Llama3-8B model requires ~16 GB of RAM. The Q2_K quantised version compresses the model weights to 2-bit precision, reducing it to ~2.7 GB.

There is a quality tradeoff — compressed models are slightly less accurate — but Q2_K is the only size that fits on my 4 GB machine alongside the OS, Python, and the RAG service.

12.2 finance_chat_Term.py — Terminal Mode

A self-contained script that runs the LLM directly in the terminal without FastAPI or Gradio. Useful for quickly testing model behaviour, debugging prompts, or running on very low-memory systems where the full stack might not fit.

12.3 finance_chat.py — Original Prototype

The very first version of the project — a single Python file that combined the LLM loading and Gradio interface. It had no agents, no RAG, no memory, and no backend separation. I kept it in the project as a record of where the project started. Comparing it to the current architecture shows how far the project evolved.

12.4 kill_ports.py — Port Cleanup

When a Python process crashes without gracefully shutting down, it can leave the port occupied. The next run then fails with "address already in use". kill_ports.py uses netstat to find and kill any process holding ports 7860, 8000, or their neighbours.

12.5 utils/data_loader.py — Knowledge Base Loader

Reads all .txt files from the docs/ directory and returns their contents as a list of strings. Used by rag_service.py at startup to build the FAISS index. Handles encoding errors gracefully so a corrupted document file doesn't crash the whole startup sequence.

[2]. How It All Connects — Data Flow

---

[2] .1 Full Request Flow (Annotated)

```
USER TYPES: 'How do I pay off my credit card faster?'
         │
         ▼
  [ui/app.py — Gradio]
  _post('/query', {user_id, message, session_id})
         │ HTTP POST to http://127.0.0.1:8000/query
         ▼
  [main.py — FastAPI]
  Validates request against ChatRequest Pydantic model
  Calls orchestrator.handle_query(request)
         │
         ▼
  [core/orchestrator.py]
  1. detect_agent('credit card') → 'debt'
  2. memory_service.get_history(session_id) → last 3 turns
  3. rag_service.retrieve_docs(message) → 3 relevant chunks from docs/
         │
         ▼
  [agents/debt_coach.py]
  prompt_service.load_prompt('debt_coach')
  Fill placeholders: {rag_context}, {memory}, {message}
  Return assembled prompt string
         │
         ▼
  [core/llm_engine.py]
  await asyncio.to_thread(llm.generate, prompt)
  ← waits 30-90 seconds →
  Returns: 'The two most effective strategies are Avalanche...'
         │
         ▼
  [core/orchestrator.py — after LLM returns]
  memory_service.save_turn(session_id, message, response)
  database_service.log_exchange(user_id, message, response, agent='debt')
  app/logger.log_interaction(user_id, message, response)
         │
         ▼
  [main.py]
  Returns ChatResponse({response, agent, sources})
         │ HTTP 200 JSON
         ▼
```

```
  [ui/app.py — Gradio]
  Appends response to chatbot history
  Displays in UI
```

13.2 Import Dependency Map

This shows which files import which other files. Reading top-to-bottom shows the dependency layers:

```
config.py               ← imported by almost everything (no dependencies)
    │
app/models.py           ← imported by main.py (Pydantic only)
app/logger.py           ← imported by orchestrator
app/prompt_service.py   ← imported by agents
utils/data_loader.py    ← imported by rag_service
    │
core/database_service.py ← imported by orchestrator
core/memory_service.py   ← imported by orchestrator
core/llm_engine.py       ← imported by orchestrator
core/rag_service.py      ← imported by orchestrator
    │
agents/budget_advisor.py ─┐
agents/debt_coach.py     ─┤ all imported by orchestrator
agents/portfolio_agent.py─┤
agents/sandbox_executor.py┘
    │
core/orchestrator.py    ← imported by main.py
    │
main.py                 ← imported by run_backend.py
ui/app.py               ← imported by run_frontend.py
```

config.py and app/models.py have no imports from within the project — they are the base layer. The agents sit in the middle layer. main.py and the UI sit at the top. This layered structure means lower layers never import from higher layers, which prevents circular imports.